

Algoritmi e Strutture Dati

Scelta della struttura dati

Alberto Montresor

Università di Trento

2026/08/15

This work is licensed under a Creative Commons
Attribution-ShareAlike 4.0 International License.



Sommario

- 1 Introduzione
 - Obiettivi
 - Definizione del problema
- 2 Cammini minimi, sorgente singola
 - Caratteristiche delle soluzioni
 - Teorema di Bellman
 - Schema generico
 - Algoritmi derivati dallo schema generico
- 3 Casi speciali – DAG
- 4 Cammini minimi, sorgente multipla
 - Floyd-Warshall
 - Chiusura transitiva
- 5 Conclusione

Obiettivo della lezione

Cosa?

Dimostrare che anche la **scelta della struttura dati** può essere considerata una tecnica algoritmica che può avere un enorme impatto sul comportamento e la complessità di un algoritmo

Come ?

Utilizziamo un esempio specifico (**problema dei cammini minimi**), costruendo uno schema generico di soluzione e ottenendo algoritmi diversi cambiando la struttura dati

Bonus track

Vedremo anche altri algoritmi per i cammini minimi, per capire quando usare ciascun approccio

Problema cammini minimi singola sorgente

Input

- Grafo orientato $G = (V, E)$
- Un nodo **sorgente** s
- Una **funzione di peso** $w : E \rightarrow R$

Definizione

Dato un cammino $p = \langle v_0, v_1, \dots, v_k \rangle$ con $k \geq 1$, il **costo del cammino** è dato da

$$w(p) = \sum_{i=1}^k w(v_{i-1}, v_i)$$

Output

Trovare, per ogni nodo $u \in V$, un cammino da s ad u **il cui costo sia minimo** tra tutti i cammini da s a u

Panoramica sul problema

Cammini minimi da singola sorgente

- **Input:** Grafo pesato, nodo radice s
- **Output:** i cammini minimi che vanno da s a tutti gli altri nodi

Cammino minimo tra una coppia di vertici

- **Input:** Grafo pesato, una coppia di vertici s, d
- **Output:** un cammino minimo fra s e d
- Si risolve il primo problema e si estrae il cammino richiesto
- Algoritmi

Panoramica sul problema

Cammini minimi tra tutte le coppie di vertici

- **Input:** Grafo pesato
- **Output:** i cammini minimi fra tutte le coppie di vertici.
- Soluzione basata su programmazione dinamica

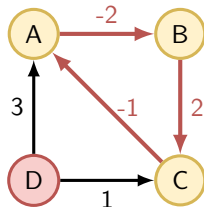
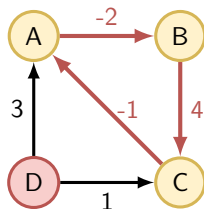
Pesi

Algoritmi diversi possono specializzarsi per particolari tipi di pesi

- Positivi / positivi+negativi
- Reali / interi

Esempio: proprietario di un TIR

- Viaggiare scarico: perdita, peso positivo
- Viaggiare carico: profitto, peso negativo



Domanda: cosa succede con i cicli negativi?

Problema cammini minimi – Sottostruttura ottima

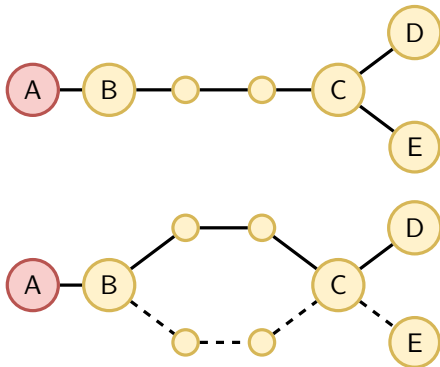
Albero dei cammini minimi

L'**albero dei cammini minimi** è un albero di copertura radicato in s avente un cammino da s a tutti i nodi raggiungibili da s .

Due cammini minimi possono avere un tratto in comune $A \rightsquigarrow C$

...

...ma non possono convergere in un nodo comune C dopo aver percorso un tratto distinto

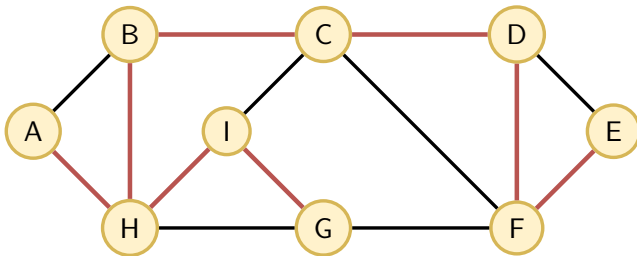


Ripasso

Albero di copertura (Spanning tree)

Dato un grafo $G = (V, E)$ non orientato e connesso, un albero di copertura di G è un sottografo $T = (V, E_T)$ tale che

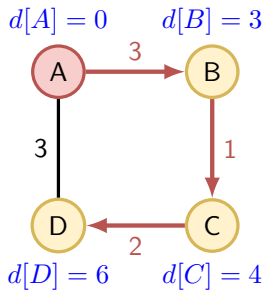
- T è un albero
- $E_T \subseteq E$
- T contiene tutti i vertici di G



Soluzione ammissibile

Soluzione ammissibile

Una soluzione **ammissibile** può essere descritta da un **albero di copertura T** radicato in s e da un **vettore di distanza d** , i cui valori $d[u]$ rappresentano il costo del cammino da s a u in T .



Rappresentazione dell'albero

Per rappresentare l'albero, utilizziamo la rappresentazione basata su vettore dei padri, così come abbiamo fatto con le visite in ampiezza/profondità.

```
printPath(NODE s, NODE d, NODE[] T)
```

```
if s == d then  
  | print s  
else if T[d] == nil then  
  | print "error"  
else  
  | printPath(s, T[d], T)  
  | print d
```

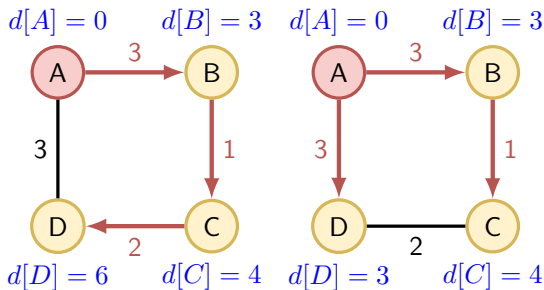
Come riconoscere una soluzione ottima?

Domanda

Abbiamo visto che una soluzione ammissibile è descritta da:

- un vettore di distanze d
- un albero T radicato in s

Ma come possiamo capire se questa soluzione è **ottima**?



Come riconoscere una soluzione ottima?

Domanda

Abbiamo visto che una soluzione ammissibile è descritta da:

- un vettore di distanze d
- un albero T radicato in s

Ma come possiamo capire se questa soluzione è **ottima**?

Idea

Se esistesse un arco (u, v) tale che $d[u] + w(u, v) < d[v]$ allora potremmo ottenere un cammino più corto verso v passando da u .

Conclusione informale

In una soluzione ottima non deve esistere nessun arco che permette di **migliorare una distanza**.

Teorema di Bellman

Teorema di Bellman

Una soluzione ammissibile T è **ottima** se e solo se:

$$d[v] = d[u] + w(u, v) \quad \forall (u, v) \in T \quad \text{Coerenza della soluzione}$$

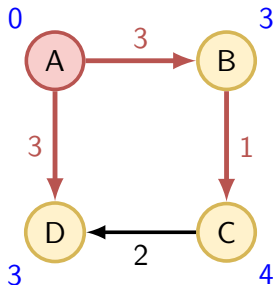
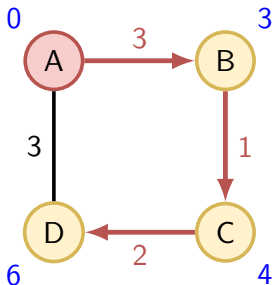
$$d[v] \leq d[u] + w(u, v) \quad \forall (u, v) \in E \quad \text{Nessun miglioramento}$$

$$d[B] = d[A] + w(A, B) \quad d[C] = d[B] + w(B, C)$$

$$d[D] = d[C] + w(C, D) \quad d[D] > d[A] + w(A, D)$$

$$d[B] = d[A] + w(A, B) \quad d[C] = d[B] + w(B, C)$$

$$d[D] = d[A] + w(A, D) \quad d[D] \leq d[C] + w(C, D)$$



Dimostrazione

Teorema di Bellman - Parte $\Rightarrow$

Se T è una soluzione ottima, allora valgono le condizioni di Bellman:

$$d[v] = d[u] + w(u, v) \quad \text{per ogni arco } (u, v) \in T$$

$$d[v] \leq d[u] + w(u, v) \quad \text{per ogni arco } (u, v) \in E$$

Sia T una soluzione ottima e sia $(u, v) \in E$.

- Se $(u, v) \in T$, allora $d[v] = d[u] + w(u, v)$
- Se $(u, v) \notin T$, allora $d[v] \leq d[u] + w(u, v)$, perché altrimenti esisterebbe nel grafo G un cammino da s a v più corto di quello in T , assurdo.

Dimostrazione

Teorema di Bellman - Parte $\Leftarrow$

Se valgono le condizioni di Bellman:

$$d[v] = d[u] + w(u, v) \quad \text{per ogni arco } (u, v) \in T$$

$$d[v] \leq d[u] + w(u, v) \quad \text{per ogni arco } (u, v) \in E$$

allora T è una soluzione ottima.

- Supponiamo per assurdo che T non sia ottimo
- Allora esiste un nodo u tale che il cammino $C_T(s, u)$ da s ad u in T non è il cammino minimo da s a u
- Allora esiste un albero ottimo T' , in cui il cammino $C_{T'}(s, u)$ ha distanza $d'[u] < d[u] \dots$
- ...dove d' il vettore delle distanze associato a T'

Dimostrazione

Teorema di Bellman - Parte $\Leftarrow$

Se valgono le condizioni di Bellman:

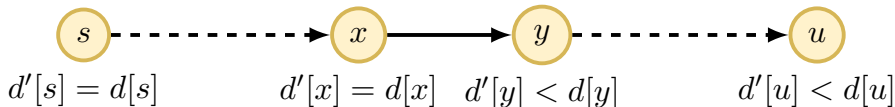
$$d[v] = d[u] + w(u, v) \quad \text{per ogni arco } (u, v) \in T$$

$$d[v] \leq d[u] + w(u, v) \quad \text{per ogni arco } (u, v) \in E$$

allora T è una soluzione ottima.

- Poichè $d'[s] = d[s] = 0$, ma $d'[u] < d[u]$, esiste un arco (x, y) in C' tale che:

- $d'[x] = d[x]$ e
- $d'[y] < d[y]$



Dimostrazione

Teorema di Bellman - Parte $\Leftarrow$

Se valgono le condizioni di Bellman:

$$d[v] = d[u] + w(u, v) \quad \text{per ogni arco } (u, v) \in T$$

$$d[v] \leq d[u] + w(u, v) \quad \text{per ogni arco } (u, v) \in E$$

allora T è una soluzione ottima.

- Per costruzione: $d'[x] = d[x]$
- Per costruzione: $d'[y] = d'[x] + w(x, y)$
- Per ipotesi: $d[y] \leq d[x] + w(x, y)$
- Combinando queste due relazioni, si ottiene:

$$d'[y] = d'[x] + w(x, y) = d[x] + w(x, y) \geq d[y]$$

Quindi $d'[y] \geq d[y]$, il che contraddice $d'[y] < d[y]$

Algoritmo prototipo – Rilassamento

(**int**[], **int**[]) **prototipoCamminiMinimi**(GRAPH G , NODE s)

% Inizializza T ad una foresta di copertura composta da nodi isolati

% Inizializza d con sovrastima della distanza ($d[s] = 0$, $d[x] = +\infty$)

while $\exists(u, v) : d[u] + G.w(u, v) < d[v]$ **do**

$d[v] = d[u] + G.w(u, v)$
 % Sostituisci il padre di v in T con u

return (T, d)

Note

- L'operazione di aggiornamento dei pesi si chiama **rilassamento**
- Se al termine dell'esecuzione qualche nodo mantiene una distanza infinita, esso non è raggiungibile
- Come implementare la condizione $\exists$?

Algoritmo generico

```
(int[], int[]) shortestPath(GRAPH  $G$ , NODE  $s$ )
```

```
int[]  $d$  = new int[0... $G.n - 1$ ]           %  $d[u]$  è la distanza da  $s$  a  $u$ 
int[]  $T$  = new int[0... $G.n - 1$ ]         %  $T[u]$  è il padre di  $u$  nell'albero  $T$ 
boolean[]  $b$  = new boolean[1... $G.n$ ]      %  $b[u]$  è true se  $u \in S$ 
foreach  $u \in G.V() - \{s\}$  do
    |  $T[u] = \mathbf{nil}$ 
    |  $d[u] = +\infty$ 
    |  $b[u] = \mathbf{false}$ 
 $T[s] = \mathbf{nil}$ 
 $d[s] = 0$ 
 $b[s] = \mathbf{true}$ 
[...]
```

Algoritmo generico

...shortestPath($\text{GRAPH } G$, $\text{NODE } s$)

(1) **DATASTRUCTURE** $S = \text{DataStructure}()$; $S.\text{add}(s)$

while not $S.\text{isEmpty}()$ **do**

(2) **int** $u = S.\text{extract}()$

$b[u] = \text{false}$

foreach $v \in G.\text{adj}(u)$ **do**

if $d[u] + G.w(u, v) < d[v]$ **then**

if not $b[v]$ **then**

(3) $S.\text{add}(v)$

$b[v] = \text{true}$

else

(4) % Azione da svolgere nel caso v sia già presente in S

$T[v] = u$

$d[v] = d[u] + G.w(u, v)$

return (T, d)

Dijkstra, 1959

Storia

- Sviluppato da Edsger W. Dijkstra nel 1956, pubblicato nel 1959
- Nella versione originale:
 - Veniva utilizzata per trovare la distanza minima fra due nodi
 - Utilizzava il concetto di coda con priorità
 - Tenete conto però che gli heap sono stati proposti nel 1964

Note

- Funziona (bene) solo con pesi non negativi
- Utilizzato in protocolli di rete come IS-IS e OSPF

Dijkstra, 1959 – Coda con priorità basata su vettore

Linea (1): Inizializzazione

- Viene creato un vettore di dimensione n
- L'indice u rappresenta il nodo u -esimo
- Le priorità vengono inizializzate ad $+\infty$
- La priorità di s è posta uguale a 0
- Costo computazionale: $\mathcal{O}(n)$

...**shortestPath**(GRAPH G , NODE s)

(1) **PRIORITYQUEUE** $Q = \text{PriorityQueue}(); Q.\text{insert}(s, 0)$

while not $Q.\text{isEmpty}()$ **do**

(2) **int** $u = Q.\text{deleteMin}()$
 $b[u] = \text{false}$
 foreach $v \in G.\text{adj}(u)$ **do**
 if $d[u] + G.w(u, v) < d[v]$ **then**
 [...]

Dijkstra, 1959 – Coda con priorità basata su vettore

Linea (2): Estrazione minimo

- Si ricerca il minimo all'interno del vettore
- Una volta trovato, si "cancella" la sua priorità
- Costo computazionale: $\mathcal{O}(n)$

...shortestPath(GRAPH G , NODE s)

(1) PRIORITYQUEUE $Q = \text{PriorityQueue}()$; $Q.\text{insert}(s, 0)$

while not $Q.\text{isEmpty}()$ **do**

(2) $u = Q.\text{deleteMin}()$
 $b[u] = \text{false}$
 foreach $v \in G.\text{adj}(u)$ **do**
 if $d[u] + G.w(u, v) < d[v]$ **then**
 $\lfloor \dots \rfloor$

return (T, d)

Dijkstra, 1959 – Coda con priorità basata su vettore

Linea (3): Inserimento in coda

- Si registra la priorità nella posizione v -esima
- Costo computazionale: $\mathcal{O}(1)$

...shortestPath(GRAPH G , NODE s)

[...]

if $d[u] + G.w(u, v) < d[v]$ **then**

if not $b[v]$ **then**

(3) | $Q.insert(v, d[u] + G.w(u, v))$
 | $b[v] = \mathbf{true}$

else

(4) | % Azione da svolgere nel caso v sia già presente in Q

$T[v] = u$

$d[v] = d[u] + G.w(u, v)$

[...]

Dijkstra, 1959 – Coda con priorità basata su vettore

Linea (4): Aggiornamento priorità

- Si aggiorna la priorità nella posizione v -esima
- Costo computazionale: $\mathcal{O}(1)$

...**shortestPath**(GRAPH G , NODE s)

[...]

if $d[u] + G.w(u, v) < d[v]$ **then**

if not $b[v]$ **then**

(3) | $Q.insert(v, d[u] + G.w(u, v))$
 | $b[v] = \mathbf{true}$

else

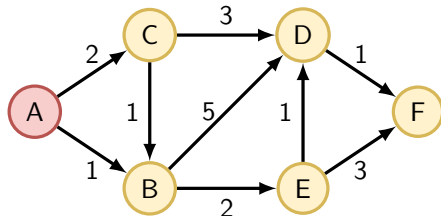
(4) | $Q.decrease(v, d[u] + G.w(u, v))$

$T[v] = u$

$d[v] = d[u] + G.w(u, v)$

[...]

Dijkstra, 1959 – Coda con priorità basata su vettore



		A	B	C	E	D	F
A	0	0	0	0	0	0	0
B	∞	1	1	1	1	1	1
C	∞	2	2	2	2	2	2
D	∞	∞	6	5	4	4	4
E	∞	∞	3	3	3	3	3
F	∞	∞	∞	∞	6	5	5

- Ogni colonna contiene lo stato del vettore d all'inizio di ogni ripetizione del ciclo `while not Q.isEmpty()`
- Ogni riga v rappresenta l'evoluzione dell'elemento $d[v]$
- La legenda delle colonne rappresenta il nodo che viene estratto

Dijkstra – Correttezza

Correttezza per pesi non negativi

- Ogni nodo raggiungibile viene estratto una volta
- Quando un nodo u viene estratto, $d[u]$ è la distanza minima da s a u

Dimostrazione per induzione sul numero k di nodi estratti

Caso base: Il primo nodo estratto è s . Poiché $d[s] = 0$ e tutti i pesi sono non negativi, qualunque cammino verso s ha peso non negativo e quindi $d[s] = 0$ è minimo. **Ipotesi induttiva:** I primi $k - 1$ nodi estratti

hanno distanza corretta.

Dijkstra – Correttezza

Passo induttivo: Sia u il k -esimo nodo estratto.

Per assurdo, supponiamo esista un cammino da s a u più corto di $d[u]$

Sia y il primo nodo di questo cammino che non è ancora stato estratto (al limite, u) e sia x il suo predecessore.

- x è già stato estratto, quindi per ipotesi induttiva $d[x]$ è corretto.
- Quando x è stato estratto, l'arco (x, y) è stato rilassato:
$$d[y] \leq d[x] + w(x, y)$$
- Ma il cammino scelto è più corto di $d[u]$, quindi
$$d[x] + w(x, y) < d[u]$$
- Ne segue $d[y] < d[u]$, quindi y avrebbe dovuto essere estratto prima di u : contraddizione

Dijkstra, 1959 – Coda con priorità basata su vettore

Costo computazionale

Riga	Costo	Ripet.
(1)	$\mathcal{O}(n)$	1
(2)	$\mathcal{O}(n)$	$\mathcal{O}(n)$
(3)	$\mathcal{O}(1)$	$\mathcal{O}(n)$
(4)	$\mathcal{O}(1)$	$\mathcal{O}(m)$

Costo totale: $\mathcal{O}(n^2)$

...shortestPath(GRAPH G , NODE s)

```

[...]
```

(1) **PRIORITYQUEUE** $Q = \text{PriorityQueue}(); Q.\text{insert}(s, 0)$

while not $Q.\text{isEmpty}()$ **do**

(2) $u = Q.\text{deleteMin}()$

$b[u] = \text{false}$

foreach $v \in G.\text{adj}(u)$ **do**

if $d[u] + G.w(u, v) < d[v]$ **then**

if not $b[v]$ **then**

(3) $Q.\text{insert}(v, d[u] + G.w(u, v))$

$b[v] = \text{true}$

else

(4) $Q.\text{decrease}(v, d[u] + G.w(u, v))$

$T[v] = u$

$d[v] = d[u] + G.w(u, v)$

return (T, d)

Johnson, 1977 – Coda con priorità basata su heap binario

Costo computazionale

Riga	Costo	Ripet.
(1)	$\mathcal{O}(n)$	1
(2)	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$
(3)	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$
(4)	$\mathcal{O}(\log n)$	$\mathcal{O}(m)$

Costo totale: $\mathcal{O}(m \log n)$

Heap binario introdotto nel 1964

...**shortestPath**(GRAPH G , NODE s)

[...]

```

(1) PRIORITYQUEUE  $Q = \text{PriorityQueue}(); Q.\text{insert}(s, 0)$ 
    while not  $Q.\text{isEmpty}()$  do
(2)    $u = Q.\text{deleteMin}()$ 
       $b[u] = \text{false}$ 
      foreach  $v \in G.\text{adj}(u)$  do
(3)         if  $d[u] + G.w(u, v) < d[v]$  then
(4)           if not  $b[v]$  then
               $Q.\text{insert}(v, d[u] + G.w(u, v))$ 
               $b[v] = \text{true}$ 
           else
               $Q.\text{decrease}(v, d[u] + G.w(u, v))$ 
               $T[v] = u$ 
               $d[v] = d[u] + G.w(u, v)$ 
    return  $(T, d)$ 
  
```

Fredman-Tarjan, 1987 – Heap di Fibonacci

Costo computazionale

Riga	Costo	Ripet.
(1)	$\mathcal{O}(n)$	1
(2)	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$
(3)	$\mathcal{O}(1)^{(*)}$	$\mathcal{O}(n)$
(4)	$\mathcal{O}(1)^{(*)}$	$\mathcal{O}(m)$

Costo: $\mathcal{O}(m + n \log n)$

(*) Costo ammortizzato

...**shortestPath**(GRAPH G , NODE s)

[...]

```

(1) PRIORITYQUEUE  $Q = \text{PriorityQueue}(); Q.\text{insert}(s, 0)$ 
while not  $Q.\text{isEmpty}()$  do
(2)    $u = Q.\text{deleteMin}()$ 
       $b[u] = \text{false}$ 
      foreach  $v \in G.\text{adj}(u)$  do
        if  $d[u] + G.w(u, v) < d[v]$  then
          if not  $b[v]$  then
(3)              $Q.\text{insert}(v, d[u] + G.w(u, v))$ 
                   $b[v] = \text{true}$ 
          else
(4)              $Q.\text{decrease}(v, d[u] + G.w(u, v))$ 
                   $T[v] = u$ 
                   $d[v] = d[u] + G.w(u, v)$ 
      return  $(T, d)$ 
  
```

Bellman-Ford-Moore, 1958 – Coda

Storia

- Proposto da **Alfonso Shimbel** nel 1955
- Pubblicato da **Lester Ford, Jr.** nel 1956
- Pubblicato da **Edward F. Moore** nel 1958
 - **Shortest path faster algorithm**
 - La versione che vediamo qui
- Pubblicato da **Richard Bellman** nel 1958
- Noto come Bellman-Ford, o Bellman-Ford-Moore

Note

- Computazionalmente più pesante di Dijkstra
- Funziona anche con archi di peso negativo

Bellman-Ford-Moore, 1958 – Coda

Linea (1): Inizializzazione

- Viene creata una coda di dimensione n
- Costo computazionale: $\mathcal{O}(1)$

...**shortestPath**(GRAPH G , NODE s)

```
(1) QUEUE  $Q = \text{Queue}(); Q.\text{enqueue}(s)$   
while not  $Q.\text{isEmpty}()$  do  
(2)    $u = Q.\text{dequeue}()$   
       $b[u] = \text{false}$   
      foreach  $v \in G.\text{adj}(u)$  do  
        if  $d[u] + G.w(u, v) < d[v]$  then  
           $\lfloor \dots \rfloor$   
return  $(T, d)$ 
```

Bellman-Ford-Moore, 1958 – Coda

Linea (2): Estrazione

- Viene estratto il prossimo elemento della coda
- Costo computazionale: $\mathcal{O}(1)$

...**shortestPath**(GRAPH G , NODE s)

```
(1) QUEUE  $Q = \text{Queue}(); Q.\text{enqueue}(s)$   
   while not  $Q.\text{isEmpty}()$  do  
(2)    $u = Q.\text{dequeue}()$   
        $b[u] = \text{false}$   
       foreach  $v \in G.\text{adj}(u)$  do  
         if  $d[u] + G.w(u, v) < d[v]$  then  
            $\lfloor \dots \rfloor$   
return  $(T, d)$ 
```

Bellman-Ford-Moore, 1958 – Coda

Linea (3): Inserimento in coda

- Si inserisce l'indice v in coda
- Costo computazionale: $\mathcal{O}(1)$

...**shortestPath**(GRAPH G , NODE s)

[...]

if $d[u] + G.w(u, v) < d[v]$ **then**

if not $b[v]$ **then**

$Q.enqueue(v)$

$b[v] = \mathbf{true}$

else

 % Azione da svolgere nel caso v sia già presente in S

$T[v] = u$

$d[v] = d[u] + G.w(u, v)$

[...]

Bellman-Ford-Moore, 1958 – Coda

Linea (4): Azione nel caso v sia già presente in S

- Sezione non necessaria

...**shortestPath**(GRAPH G , NODE s)

[...]

if $d[u] + G.w(u, v) < d[v]$ **then**

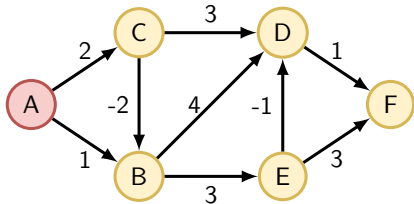
(3) **if not** $b[v]$ **then**
 $Q.enqueue(v)$
 $b[v] = \mathbf{true}$

$T[v] = u$

$d[v] = d[u] + G.w(u, v)$

[...]

Bellman-Ford-Moore, 1958 – Coda



- La prima riga contiene l'elemento estratto dalla coda
- L'ultima riga contiene lo stato della coda

		A	B	C	D	E	B	F	D	E	D
A	0	0	0	0	0	0	0	0	0	0	0
B	∞	1	1	0	0	0	0	0	0	0	0
C	∞	2	2	2	2	2	2	2	2	2	2
D	∞	∞	5	5	5	3	3	3	3	2	2
E	∞	∞	4	4	4	4	3	3	3	3	3
F	∞	∞	∞	∞	6	6	6	6	4	4	3
Q	A	BC	CDE	DEB	EBF	BFD	FDE	DE	E	D	F

Bellman-Ford-Moore, 1958 – Coda

Passata - definizione ricorsiva

- Per $k = 0$, la zeresima passata consiste nell'estrazione del nodo s dalla coda S ;
- Per $k > 0$, la k -esima passata consiste nell'estrazione di tutti i nodi presenti in S al termine della passata $k - 1$ -esima.

Correttezza – intuizione

- Al termine della passata k , i vettori T e d descrivono i cammini minimi composti da al più k archi
- Al termine della passata $n - 1$, i vettori T e d descrivono i cammini minimi (composti da al più $n - 1$ archi)

Bellman-Ford-Moore, 1958 – Coda

Costo computazionale

Riga	Costo	Ripet.
(1)	$\mathcal{O}(1)$	1
(2)	$\mathcal{O}(1)$	$\mathcal{O}(n^2)$
(3)	$\mathcal{O}(1)$	$\mathcal{O}(nm)$

Costo: $\mathcal{O}(nm)$

Ogni nodo può essere inserito ed estratto al massimo $n - 1$ volte

```
shortestPath(GRAPH  $G$ , NODE  $s$ )
```

```

(1) QUEUE  $Q = \text{Queue}(); Q.\text{enqueue}(s)$ 
while not  $Q.\text{isEmpty}()$  do
  (2)    $u = Q.\text{dequeue}()$ 
          $b[u] = \text{false}$ 
         foreach  $v \in G.\text{adj}(u)$  do
           if  $d[u] + G.w(u, v) < d[v]$  then
             if not  $b[v]$  then
               (3)    $Q.\text{enqueue}(v)$ 
                        $b[v] = \text{true}$ 
                        $T[v] = u$ 
                        $d[v] = d[u] + G.w(u, v)$ 
         return  $(T, d)$ 
```

Cammini minimi su DAG

Osservazione

I cammini minimi in un DAG sono sempre ben definiti; anche in presenza di pesi negativi, non esistono cicli negativi

Come risolvere il problema?

È possibile rilassare gli archi in ordine topologico, una volta sola. Non essendoci cicli, non c'è modo di tornare su un nodo già visitato e abbassare il valore del suo campo d

Algoritmo

- Si utilizza l'ordinamento topologico

Cammini minimi su DAG

```
(int[], int[]) shortestPath(GRAPH  $G$ , NODE  $s$ )
```

```
int[]  $d$  = new int[1 ...  $G.n$ ]
```

% $d[u]$ è la distanza da s a u

```
int[]  $T$  = new int[1 ...  $G.n$ ]
```

% $T[u]$ è il padre di u nell'albero T

```
foreach  $u \in G.V() - \{s\}$  do
```

```
     $T[u] = \text{nil}; d[u] = +\infty$ 
```

```
 $T[s] = \text{nil}; d[s] = 0$ 
```

```
STACK  $S$  = topsort( $G$ )
```

```
while not  $S.\text{isEmpty}()$  do
```

```
     $u = S.\text{pop}()$ 
```

```
    foreach  $v \in G.\text{adj}(u)$  do
```

```
        if  $d[u] + G.w(u, v) < d[v]$  then
```

```
             $T[v] = u$ 
```

```
             $d[v] = d[u] + G.w(u, v)$ 
```

```
return ( $T, d$ )
```

Riassunto

Complessità: quale preferire?

BFS	$\mathcal{O}(m + n)$	Senza pesi
Dijkstra	$\mathcal{O}(n^2)$	Pesi positivi, grafi densi
Johnson	$\mathcal{O}(m \log n)$	Pesi positivi, grafi sparsi
Fredman-Tarjan	$\mathcal{O}(m + n \log n)$	Pesi positivi, grafi densi, dimensioni molto grandi
Bellman-Ford	$\mathcal{O}(mn)$	Pesi negativi
	$\mathcal{O}(m + n)$	DAG
Bernstein, Nanongkai Wulff-Nilsen (2022)	$\mathcal{O}(m \log^8 n \log W)$	Pesi negativi, interi

Cammini minimi, sorgente multipla

Possibili soluzioni

Input	Complessità	Approccio
Pesi positivi, grafo denso	$\mathcal{O}(n \cdot n^2)$	Applicazione ripetuta dell'algoritmo di Dijkstra
Pesi positivi, grafo sparso	$\mathcal{O}(n \cdot (m \log n))$	Applicazione ripetuta dell'algoritmo di Johnson
Pesi negativi	$\mathcal{O}(n \cdot nm)$	Applicazione ripetuta di Bellman-Ford, sconsigliata
Pesi negativi, grafo denso	$\mathcal{O}(n^3)$	Algoritmo di Floyd e Warshall
Pesi negativi, grafo sparso	$\mathcal{O}(nm \log n)$	Algoritmo di Johnson per sorgente multipla , non lo vediamo

Floyd-Warshall, 1962

Cammini minimi k -vincolati

Sia k un valore in $\{0, \dots, n\}$. Diciamo che un cammino p_{xy}^k è un **cammino minimo k -vincolato** fra x e y se esso ha il costo minimo fra tutti i cammini fra x e y che non passano per nessun vertice in $v_k, \dots, v_{n-1}$ (x e y sono esclusi dal vincolo).

Note

Assumiamo (come abbiamo sempre fatto) che esista un ordinamento fra i nodi del grafo $v_0, v_1, \dots, v_{n-1}$.

Domande

- A cosa corrisponde p_{xy}^0 ?
- A cosa corrisponde p_{xy}^n ?

Floyd-Warshall, 1962

Distanza k -vincolata

Denotiamo con $d^k[x][y]$ il costo totale del cammino minimo k -vincolato fra x e y , se esiste.

$$d^k[x][y] = \begin{cases} w(p_{xy}^k) & \text{se esiste } p_{xy}^k \\ +\infty & \text{altrimenti} \end{cases}$$

Domande

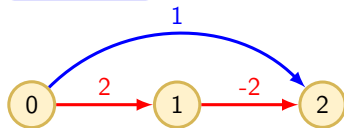
- A cosa corrisponde $d^0[x][y]$?
- A cosa corrisponde $d^n[x][y]$?

Floyd-Warshall, 1962

Formulazione ricorsiva

$$d^k[x][y] = \begin{cases} w(x, y) & k = 0 \\ \min(d^{k-1}[x][y], d^{k-1}[x][k-1] + d^{k-1}[k-1][y]) & k > 0 \end{cases}$$

Esempio



$$d^0[0][2] = 1$$

$$d^1[0][2] = 1$$

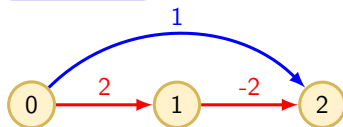
$$\begin{aligned} d^2[0][2] &= \min(d^1[0][2], d^1[0][1] + d^1[1][2]) \\ &= \min(1, 0) = 0 \end{aligned}$$

Floyd-Warshall, 1962

Matrice dei padri

Oltre a definire la matrice d , calcoliamo una matrice T dove $T[x][y]$ rappresenta il predecessore di y nel cammino più breve da x a y .

Esempio



$$T[0][1] = 0$$

$$T[1][2] = 1$$

$$T[0][2] = 1$$

Floyd-Warshall, programmazione dinamica

(int[][] , int[][]) floydWarshall(G_{GRAPH} G)

int[][] $d = \text{new int}[0 \dots n - 1][0 \dots n - 1]$

int[][] $T = \text{new int}[0 \dots n - 1][0 \dots n - 1]$

foreach $u, v \in G.V()$ **do**

$d[u][v] = +\infty$
 $T[u][v] = \text{nil}$

foreach $u \in G.V()$ **do**

$d[u][u] = 0$

foreach $u \in G.V()$ **do**

foreach $v \in G.\text{adj}(u)$ **do**
 $d[u][v] = G.w(u, v)$
 $T[u][v] = u$

Floyd-Warshall, programmazione dinamica

```
(int[[]], int[[]]) floydWarshall( $\text{GRAPH } G$ )
```

```
[...]
```

```
for  $k = 1$  to  $G.n$  do
```

```
    foreach  $u \in G.V()$  do
```

```
        foreach  $v \in G.V()$  do
```

```
            if  $d[u][k-1] + d[k-1][v] < d[u][v]$  then
```

```
                 $d[u][v] = d[u][k-1] + d[k-1][v]$ 
```

```
                 $T[u][v] = T[k-1][v]$ 
```

```
return ( $d, T$ )
```

Chiusura transitiva (Algoritmo di Warshall)

Chiusura transitiva

La chiusura transitiva $G^* = (V, E^*)$ di un grafo $G = (V, E)$ è il grafo orientato tale che $(u, v) \in E^*$ se e solo esiste un cammino da u a v in G .

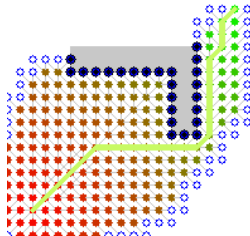
Supponendo di avere il grafo G rappresentato da una matrice di adiacenza M , la matrice M^n rappresenta la matrice di adiacenza di G^* .

Formulazione ricorsiva

$$M^k[x][y] = \begin{cases} y \in G.\text{adj}(x) & k = 0 \\ M^{k-1}[x][y] \text{ or } (M^{k-1}[x][k-1] \text{ and } M^{k-1}[k-1][y]) & k > 0 \end{cases}$$

Conclusione

- Abbiamo visto una panoramica dei più importanti algoritmi per la ricerca dei cammini minimi
- Ulteriori possibilità:
 - A*, un algoritmo che utilizza euristiche per velocizzare la ricerca
 - Algoritmi specializzati per reti stradali



https://en.wikipedia.org/wiki/A*_search_algorithm#/media/File:Astar_progress_animation.gif